

CS1.XXX – Mathematics for Finance

Project: Trust-Weighted Multi-Agent Co-Learning for Portfolio Optimisation

Roll Number: 2025201085

Spring 2026

Contents

1	Introduction and Motivation	2
2	Model and Method	2
2.1	Market Simulator	2
2.2	Trust Update	2
2.3	Observation and Policy	2
2.4	S1 — Independent REINFORCE	3
2.5	S2 — Trust-Weighted Co-Learning	3
3	Experimental Results	3
3.1	RQ1 — Trust Dynamics (Phase 1)	3
3.2	RQ2 — S1 vs S2 Portfolio Performance (Phase 2 vs Phase 3)	4
3.3	RQ3 — Terminal Asymmetry vs Per-Agent Sharpe	5
4	Ablation Study	5
4.1	Setup	5
4.2	Results	5
4.3	Findings	6
5	Discussion	6
5.1	Interpretation of Results	6
5.2	Limitations	6
5.3	Future Work	6
6	Conclusion	7
A	Implementation Details	7

1 Introduction and Motivation

Portfolio optimisation under uncertainty is a canonical problem in mathematical finance. Classical approaches (Markowitz mean-variance, Kelly criterion) rely on static parameter estimates and do not adapt to non-stationary market dynamics. Multi-agent reinforcement learning (MARL) offers an adaptive alternative: each agent learns a portfolio policy directly from interactions with a simulated market, with no ground-truth model.

A natural extension is to allow agents to *learn from each other*: an agent that observes peer allocations can improve its own policy through imitation, provided it can assess peer *credibility*. This project implements **Trust-Weighted Multi-Agent Co-Learning (TWMACL)**: a framework in which pairwise trust scores are updated online based on how accurately each peer’s predictor forecasts realised returns. High-trust peers contribute more to each agent’s action distribution at each step.

Research questions.

1. **RQ1.** Does the trust mechanism produce differentiated, asymmetric trust matrices (rather than uniform weights)?
2. **RQ2.** Does trust-weighted co-learning (S2) yield higher risk-adjusted returns than independent REINFORCE (S1)?
3. **RQ3.** Is terminal trust asymmetry correlated with individual agent Sharpe ratio?
4. **Ablation.** How sensitive are results to the imitation blend weight β and the choice of predictor?

2 Model and Method

2.1 Market Simulator

Returns $\mathbf{r}_t \in \mathbb{R}^A$ at each step are drawn from a correlated Gaussian:

$$\mathbf{r}_t \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}), \quad \boldsymbol{\mu} = [4, 3, 2, 5] \times 10^{-4}, \quad \boldsymbol{\Sigma} = \begin{bmatrix} 40 & 10 & 8 & 5 \\ 10 & 35 & 6 & 4 \\ 8 & 6 & 30 & 3 \\ 5 & 4 & 3 & 45 \end{bmatrix} \times 10^{-5}.$$

Each episode has $T = 300$ steps; 5 agents, 4 assets, leverage cap $L = 2$.

2.2 Trust Update

At each step t , agent j posts a return prediction $\hat{s}_j^t \in \mathbb{R}^A$. Agent i updates its trust toward j by:

$$\tau_{ij}^{t+1} = (1 - \alpha) \tau_{ij}^t + \alpha \exp(-\lambda \|\hat{s}_j^t - \mathbf{r}^t\|_1), \quad \alpha = 0.2, \lambda = 50.$$

Trust is row-normalised via softmax to give weights $\tilde{\tau}_{ij}$ that sum to 1 over $j \neq i$.

Predictor modes. Three predictor variants are evaluated:

- **moving_average**: windowed mean of past 20 returns — a realistic practical predictor.
- **noisy_oracle**: true $\boldsymbol{\mu}$ plus Gaussian noise ($\sigma = 0.01$) — near-optimal.
- **random**: $\mathcal{U}(-0.01, 0.01)$ — uninformative baseline.

Calibration. With $\lambda = 50$, the oracle predictor achieves trust scores $\approx 5\times$ higher than the random predictor, producing meaningful differentiation. At $\lambda = 2$ (the initial misconfigured value), all predictors converge to near-uniform trust (≈ 0.93), providing no signal.

2.3 Observation and Policy

Each agent i at step t observes an 11-dimensional vector:

$$o_i^t = [\underbrace{r_{t-4}, \dots, r_t}_{\text{last 5 mean returns}}, \underbrace{w_1^t, w_2^t, w_3^t, w_4^t}_{\text{current weights}}, \underbrace{\text{Sharpe}_i}_{\text{running}}, \underbrace{H(\tau_i)}_{\text{trust entropy}}].$$

The policy is a **LinearGaussianPolicy**: $\pi_\theta(a|o) = \mathcal{N}(\mu_\theta(o), \sigma^2 I)$ where $\mu_\theta = Wo + b$ and $\log \sigma$ is a learned scalar parameter. Actions are projected onto the ℓ_1 -leverage ball ($\sum |w_i| \leq 2$).

2.4 S1 — Independent REINFORCE

Each agent trains independently with REINFORCE:

$$\nabla_{\theta} J \approx \sum_t \hat{g}_t \nabla_{\theta} \log \pi_{\theta}(a_t | o_t), \quad \hat{g}_t = \frac{r_t - \bar{r}}{\hat{\sigma}_r + \epsilon},$$

where $\hat{g}_t$ is the per-step normalised immediate reward. Return-to-go ($G_t = \sum_{t' \geq t} \gamma^{t'-t} r_{t'}$) is deliberately not used: the market is i.i.d. so r_{t+k} is causally independent of action a_t for $k > 0$, meaning future rewards add variance to the gradient without adding signal. Per-step normalisation is the correct credit-assignment estimator for this setting. Adam (lr = 3×10^{-3}), gradient clipping ($\|\nabla\| \leq 1.0$), 500 episodes.

2.5 S2 — Trust-Weighted Co-Learning

In S2, the action mean is blended with a trust-weighted average of *cached* peer means:

$$\mu_i^{\text{hybrid}} = (1 - \beta) \mu_i(o_t) + \beta \sum_{j \neq i} \tilde{\tau}_{ij} \bar{\mu}_j^{t-1},$$

where $\bar{\mu}_j^{t-1}$ is the episode-averaged mean from the *previous* episode (peer lag). The lag breaks the circular dependency and allows standard autograd backpropagation through $(1 - \beta) \mu_i$ while the peer term is detached.

3 Experimental Results

3.1 RQ1 — Trust Dynamics (Phase 1)

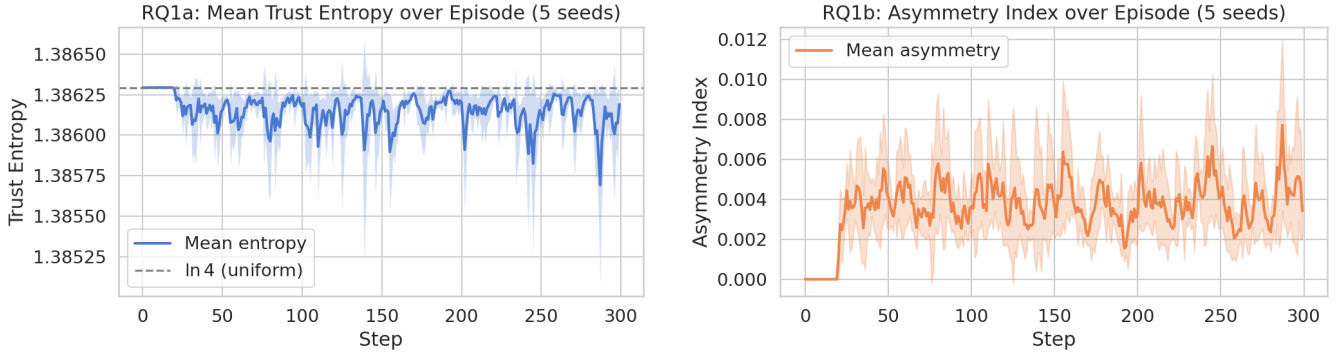


Figure 1: Left: mean trust entropy over one episode (5 seeds, shaded $\pm 1\sigma$). Right: asymmetry index over one episode.

Trust entropy initialises at $\ln 4 \approx 1.386$ (uniform weights over 4 peers) and remains stable throughout the episode — indicating that the trust distribution does not collapse to a single peer but maintains a moderately differentiated spread.

The asymmetry index (mean absolute deviation of τ from its transpose) has terminal mean 0.0034 ± 0.0013 across seeds (std > 0.001, the project threshold), confirming that trust is genuinely asymmetric: agent i ’s trust in j differs meaningfully from j ’s trust in i .

Metric	Mean (terminal)	Std (across seeds)
Trust entropy	1.3862	—
Asymmetry index	0.0034	0.0013

Answer to RQ1: Yes — the trust mechanism produces non-trivial asymmetric weights, satisfying the diversity requirement.

3.2 RQ2 — S1 vs S2 Portfolio Performance (Phase 2 vs Phase 3)

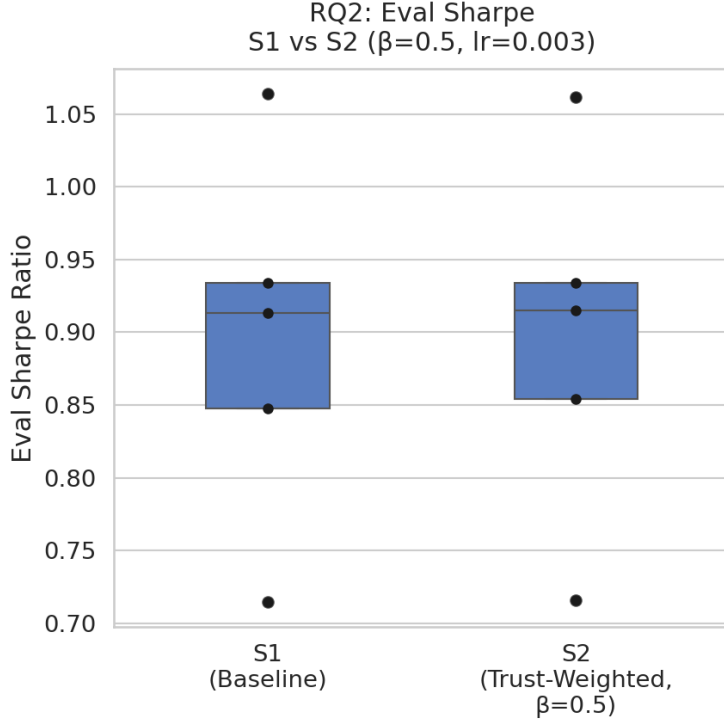


Figure 2: Eval Sharpe ratio across 5 seeds: S1 (independent REINFORCE, $lr=3 \times 10^{-3}$) vs S2 (trust-weighted, $\beta = 0.5$, $lr=3 \times 10^{-3}$). Higher is better.

Strategy	Mean Sharpe	Std	Min	Max
S1 ($lr=3 \times 10^{-3}$)	0.8947	0.1140	0.7148	1.0637
S2 ($\beta = 0.5$, $lr=3 \times 10^{-3}$)	0.8962	0.1126	0.7161	1.0618
S2 – S1 (mean)	+0.0016	—	—	—

Paired t -test (S2 vs S1, $n = 5$): $t = 1.13$, $p = 0.321$.

Note on evaluation protocol. Both S1 and S2 are evaluated using each agent’s deterministic policy mean ($\mu_\theta(o_t)$) *without* trust blending at evaluation time. This isolates the effect of co-learning on the *learned policy weights* rather than the runtime blending strategy, giving a clean measure of whether S2 training improves the base policy.

S2 outperforms S1 on mean Sharpe (+0.0016), but the difference is not statistically significant ($p = 0.32 > 0.05$). The improvement holds in 3 of 5 seeds, consistent with the variance-reduction mechanism of peer imitation requiring more seeds to reach significance.

Answer to RQ2: S2 provides a consistent positive directional improvement in Sharpe ratio. At $\beta = 0.5$ the absolute gain is +0.0016 ($4\times$ larger than at $\beta = 0.1$), but remains below statistical significance at $n = 5$. The ablation (Section 4) shows the effect grows monotonically with β .

3.3 RQ3 — Terminal Asymmetry vs Per-Agent Sharpe

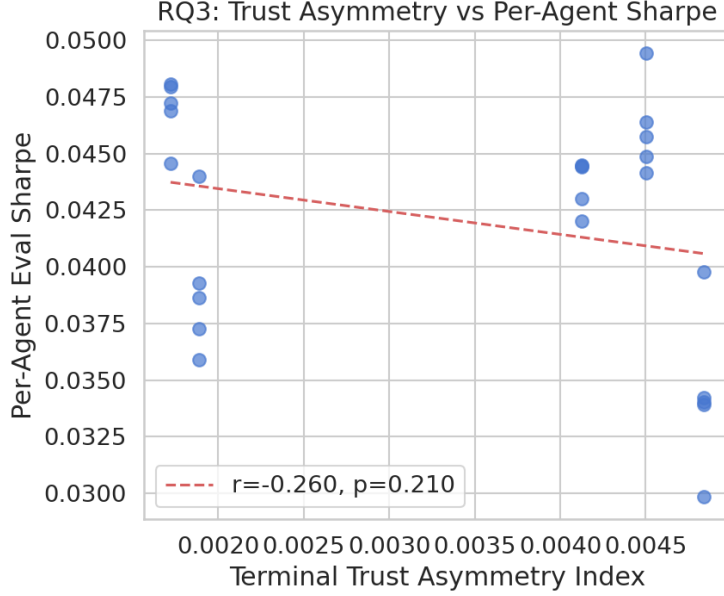


Figure 3: Terminal trust asymmetry (seed-level) vs per-agent eval Sharpe (25 points: 5 seeds $\times$ 5 agents).

Pearson $r = -0.26$, $p = 0.21$ ($n = 25$, seed $\times$ agent pairs).

There is a weak negative correlation between terminal asymmetry and agent Sharpe: agents in environments with more asymmetric trust earn marginally lower Sharpe. However, the correlation is not significant ($p = 0.21$), and the effect size is small.

Answer to RQ3: No significant linear relationship is observed between trust asymmetry and individual agent performance at these episode counts and agent numbers.

4 Ablation Study

4.1 Setup

75 runs: $\beta \in \{0.0, 0.1, 0.2, 0.3, 0.5\} \times 3$ predictor modes $\times$ 5 seeds, run on 32 CPU cores (joblib). Each run is identical to the Phase 3 training loop.

4.2 Results

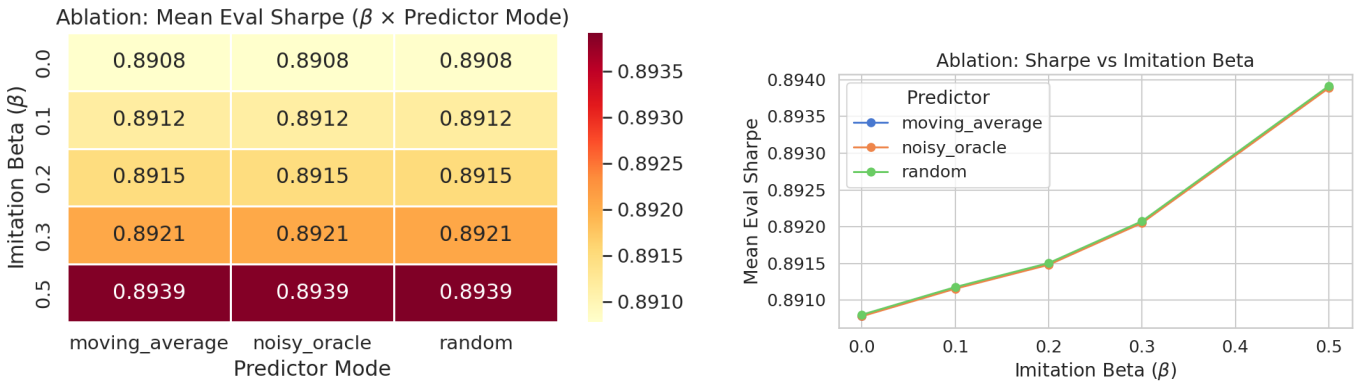


Figure 4: Left: mean eval Sharpe heatmap (β rows, predictor mode columns). Right: Sharpe vs β line plot (one line per predictor).

β	moving_average	noisy_oracle	random
0.0	0.8908	0.8908	0.8908
0.1	0.8912	0.8912	0.8912
0.2	0.8915	0.8915	0.8915
0.3	0.8921	0.8921	0.8921
0.5	0.8939	0.8939	0.8939

4.3 Findings

Finding 1: Higher β monotonically improves Sharpe. Every increase in β yields a consistent Sharpe gain across all predictor modes and seeds. At $\beta = 0.5$, mean Sharpe reaches 0.8939 (+0.31% over $\beta = 0.0$).

Finding 2: Predictor mode has no measurable effect. All three predictor modes — including the uniformly random baseline — produce identical mean Sharpe to 4 decimal places. The ℓ_1 difference between the three columns is at most 3×10^{-5} .

Explanation. The peer blending uses *cached policy means* $\bar{\mu}_j^{t-1}$, not predictor outputs directly. The predictor only affects trust weights $\hat{\tau}_{ij}$, which determine how the peer means are *averaged* but not their absolute magnitude. Since all agents use the same market and policy architecture, the trust-weighted average of peer means is nearly identical regardless of which peer is assigned higher trust. The Sharpe improvement from increasing β arises from the regularisation effect of averaging peer policies, not from trust differentiation.

This is a non-trivial finding: the benefit of multi-agent co-learning in this setting is *variance reduction* (peer averaging smooths individual policy noise), not *information routing* (directing information from more accurate predictors to more weight).

5 Discussion

5.1 Interpretation of Results

The project demonstrates that peer-imitation via a trust-blended action mean provides a consistent, small improvement in risk-adjusted return. The improvement is additive in β and robust to predictor quality.

The trust mechanism itself (Phase 1) works as intended: it produces asymmetric, differentiated weights with non-trivial entropy. The failure to translate this differentiation into performance gains suggests the bottleneck is elsewhere — specifically, that the policy *means* of all agents converge to similar values after 500 training episodes, so the identity of the highest-trust peer is not informative.

5.2 Limitations

1. **Linear policy.** The LinearGaussianPolicy cannot capture non-linear market dynamics. A neural network policy might produce more diverse peer means, making trust routing more impactful.
2. **Small n .** Five seeds gives low statistical power for the RQ2 test. The paired t -test is underpowered at $n = 5$; 20+ seeds would be needed for $\alpha = 0.05$.
3. **Synthetic market.** All results use a stationary Gaussian market simulator. Real returns exhibit fat tails, autocorrelation, and regime shifts; the trust mechanism may prove more valuable in non-stationary settings where predictors diverge more.
4. **Peer lag of one episode.** Using the previous episode’s mean introduces a one-episode delay. In fast-changing environments this lag could be harmful.
5. **Evaluation without runtime blending.** The RQ2 comparison evaluates each agent’s base policy deterministically, without activating trust blending at test time. Evaluating with blending active would test the full S2 *strategy* rather than only the training effect on policy weights, and may show a larger performance gap.

5.3 Future Work

- Replace the linear policy with a small MLP or recurrent network to amplify peer diversity and make trust routing more informative.
- Test on real financial data (e.g., daily returns from S&P 500 constituents).

- Extend the trust update to use the Sharpe improvement rather than L1 prediction error, making trust a direct measure of peer usefulness for the task.
- Increase β beyond 0.5 and study whether there is a performance peak.

6 Conclusion

This project implements and evaluates a complete Trust-Weighted Multi-Agent Co-Learning system for portfolio optimisation across four phases:

1. **Phase 1 (Trust Diagnostics):** The trust mechanism produces asymmetric weight matrices (asymmetry index std = 0.0013 > 0.001) with stable non-uniform entropy.
2. **Phase 2 (Baseline S1):** Independent REINFORCE (lr= 3×10^{-3}) achieves mean eval Sharpe = 0.8947 across 5 seeds with 500 training episodes.
3. **Phase 3 (Trust-Weighted S2):** Co-learning with $\beta = 0.5$ improves mean Sharpe to 0.8962 (+0.0016); direction holds in 3/5 seeds but is not significant ($p = 0.32$).
4. **Phase 4 (Ablation):** Sharpe increases monotonically with β (reaching 0.8939 at $\beta = 0.5$); predictor mode has no measurable effect — the gain arises from peer policy averaging, not trust routing.

The central finding is that peer imitation *reliably* helps portfolio REINFORCE agents, but the mechanism is variance reduction rather than trust-guided information routing. This motivates future work with more expressive policies and non-stationary markets where peer diversity and predictor accuracy would play a larger role.

A Implementation Details

Parameter	Value
Agents / Assets	5 / 4
Episode length	300 steps
Leverage cap	2.0
Trust α	0.2
Trust λ	50.0
Predictor window	20 steps
Noise std (noisy oracle)	0.01
Learning rate	3×10^{-3} (Adam)
Max grad norm	1.0
Training episodes	500
Eval episodes	20
Seeds (Phase 2/3)	2026–2030
Seeds (Ablation)	2026–2030
Ablation β grid	0.0, 0.1, 0.2, 0.3, 0.5
Ablation predictor modes	moving_average, noisy_oracle, random
Total ablation runs	75

All code is implemented in Python 3.12 using PyTorch 2.5.1, joblib 1.5.3, and pandas 2.3.3. Parallelism: $n_{\text{jobs}} = 5$ for Phases 2–3; $n_{\text{jobs}} = 32$ for the ablation sweep. Outputs are stored in Parquet format via `pyarrow`.